

บทที่ 4

ASP .NET

4.1 ASP คืออะไร

ASP (Active Service Pages) เป็นการเขียนโปรแกรมเพื่อประมวลผลคำสั่งบนเว็บเซิร์ฟเวอร์ ก่อนจะส่งผลลัพธ์กลับมายังเบราว์เซอร์ที่ร้องขอข้อมูลไป

เหตุผลที่ต้องมีการประมวลผลข้อมูลบนเว็บเซิร์ฟเวอร์ ก็คือ แม้ว่าเราสามารถเขียนโปรแกรมเพื่อทำตามเงื่อนไขบางอย่างได้ทางฝั่งไคลเอนต์ หรือฝั่งของผู้ใช้ โดยใช้ภาษาสคริปต์ เช่น JavaScript, VBScript แต่ก็ยังสามารถทำได้ภายในขอบเขตที่จำกัดเท่านั้น เหตุผลสำคัญประการหนึ่งของการพัฒนาเว็บไซต์ในปัจจุบัน คือ จำเป็นต้องใช้ข้อมูลการพัฒนา โดยที่ข้อมูลเหล่านี้จะมีปริมาณค่อนข้างมากตามขนาดของเว็บไซต์ ซึ่งการที่จะเคลื่อนย้ายข้อมูลเหล่านี้มายังไคลเอนต์แล้วประมวลที่นี้ย่อมเป็นไปได้ยาก นอกจากนี้แล้วความต้องการข้อมูลของผู้ใช้แต่ละคนจะมีเงื่อนไขที่ต่างกันออกไป ดังนั้นจึงจำเป็นต้องมีกระบวนการในการคัดเลือก และประมวลผลข้อมูลเหล่านั้นก่อนที่จะส่ง ไปให้กับผู้ใช้ ซึ่งการสร้างเว็บเพจแบบไดนามิก (Dynamic) นี้จะช่วยให้เราสามารถพัฒนาเว็บไซต์ในรูปแบบที่หลากหลาย และสามารถตอบสนองต่อผู้ใช้ได้อย่างสมบูรณ์มากยิ่งขึ้น

4.2 การเปลี่ยนแปลงจาก ASP ไปสู่ ASP .NET

เนื่องจากอินเทอร์เน็ตนั้นเติบโตขึ้นทุกวัน และไม่ได้จำกัดอยู่เฉพาะบนเครื่องพีซีเท่านั้น แต่ยังรวมไปถึงอุปกรณ์พกพาอย่าง มือถือ, PDA, PocketPC เป็นต้น ด้วยเหตุนี้ไมโครซอฟต์จึงต้องการให้ ASP สามารถรองรับการทำงานร่วมกับอุปกรณ์เหล่านี้ได้ด้วย

ก่อนหน้านี้ที่จะมาเป็น ASP .NET ไมโครซอฟต์ได้พัฒนา ASP มาแล้ว 3 เวอร์ชัน ซึ่งแต่ละเวอร์ชันก็ได้รับความนิยมอย่างดีจากผู้ใช้ทั่วโลก แต่อย่างไรก็ตาม ASP เวอร์ชันก่อนๆ นั้นยังมีข้อจำกัดอยู่ เช่น ต้องอาศัยภาษา HTML เป็นหลักในการเขียนโปรแกรม แล้วแทรกด้วยสคริปต์ของ VBScript ทำให้มีข้อจำกัดในการพัฒนาโปรแกรมเพราะเราไม่สามารถดึงความสามารถของภาษามาใช้ได้อย่างเต็มที่ ในเวอร์ชัน .NET จึงได้เปลี่ยนจากการใช้ VBScript มาเป็นภาษาที่สามารถขยายขีดความสามารถได้อย่างเต็มที่อย่าง VB .NET หรือ ซี-ชาร์ป (C#) ให้ผู้ใช้เลือกตามความถนัด

นอกจากนี้แล้ว ASP .NET ยังสร้างสิ่งอำนวยความสะดวกมากมายมาช่วยให้เราสามารถเขียนโปรแกรมได้ง่าย สะดวกรวดเร็ว และช่วยลดข้อผิดพลาดลงได้มาก ซึ่งต่อไปแม้ว่าผู้เขียนโปรแกรมจะไม่มีพื้นฐานความรู้ทางด้าน Programming มากนักก็สามารถสร้างเว็บไซต์ของตนเองได้แล้ว

การเปลี่ยนแปลงที่สำคัญอีกอย่างหนึ่ง คือเปลี่ยนจากการแปลภาษาสคริปต์ทีละบรรทัด (interpret) ใน ASP แบบเดิม มาเป็นการคอมไพล์โค้ด ASP .NET ให้เป็นรูปแบบ IL (Intermediate Language) ตรงนี้เองที่

สนับสนุนความจริงที่ว่า ASP .NET จะเขียนด้วยภาษาอะไรก็ได้ที่สามารถคอมไพล์เป็นรูปแบบ IL (นั่นคือ CLR ยอมรับ) ด้วยเหตุนี้ เราสามารถกล่าวได้ว่า ASP .NET ก็คือ ASP เวอร์ชันที่ถูกคอมไพล์นั่นเอง

นักพัฒนาที่ใช้ Visual Basic มักจะคุ้นเคยกับความสะดวกสบายในการเขียนโปรแกรมโดยใช้ฟอร์ม และคอนโทรลซึ่งมีรูปแบบที่สะดวกโดยลากสิ่งที่ต้องการลงมาไว้บนฟอร์มและเขียนโค้ดโดยใช้การเขียนโปรแกรมแบบ event-handling เนื่องจากวิธีนี้เป็นวิธีที่ไม่ยุ่งยาก จึงได้ถูกนำมาใช้ใน ASP.NET

ASP.NET ทำให้การพัฒนาเว็บไซต์สะดวกขึ้นด้วยการเขียนโปรแกรมโดยใช้ฟอร์ม ใน ASP.NET เราเรียกฟอร์มเหล่านี้ว่า เว็บฟอร์ม (Web Forms) ซึ่งนำมาแทนที่หน้า ASP การเขียนโปรแกรมแบบเว็บฟอร์มนี้เป็นแบบ event base เช่นเดียวกับใน visual basic นอกจากนี้ ASP.NET ยังแยกส่วนระหว่างส่วนแอปพลิเคชัน และส่วนในการแสดงผลอีกด้วย

ASP.NET เกี่ยวข้องกับรูปแบบการเขียนโปรแกรม ASP โดยมีคุณสมบัติพิเศษเพิ่มเติมขึ้นมา ดังนี้

- มีการแยกส่วนเด็ดขาดระหว่างส่วนโค้ดการประมวลผลของแอปพลิเคชันและส่วนแสดงผล (HTML markup) จึงไม่มีความสับสนในโค้ดทั้งสองส่วนอีก
- มีกลุ่มของเซิร์ฟเวอร์คอนโทรลจำนวนมากให้เลือกใช้ ซึ่งจะถูกละเปลี่ยนเป็น HTML โดยอัตโนมัติ
- มีการจัดการสถานะโดยใช้ฮิวส์เซสชัน
- เป็นรูปแบบการเขียนโปรแกรมแบบ event-base ทางฝั่งเซิร์ฟเวอร์ ทำให้ง่ายขึ้น
- สามารถเขียนโค้ดส่วนแอปพลิเคชันโดยใช้ภาษาใดใน Microsoft .NET ก็ได้ เช่น ภาษา VB, C# เป็นต้น โค้ดของแอปพลิเคชันบนฝั่งเซิร์ฟเวอร์จะถูกคอมไพล์เพื่อประสิทธิภาพการทำงานที่ดีกว่า
- มี Visual Studio.NET เป็นเครื่องมืออำนวยความสะดวกในการพัฒนาเว็บฟอร์ม

สรุปข้อดีของ ASP .NET

- เขียนได้ง่าย และดูแลรักษาได้ง่าย เนื่องจาก ASP .NET มีโครงสร้างที่จะช่วยทำให้การนำโค้ดกลับมาใช้ใหม่ และการใช้งานโค้ดร่วมกันทำได้ง่ายขึ้น
- การส่งมอบ (Deployment) แอปพลิเคชันทำได้สะดวกมากขึ้น ซึ่งจากในเวอร์ชันเดิมของ ASP จะมีความยุ่งยากมาก สำหรับระบบที่มีคอมโพเนนต์อยู่มากมาย แต่สำหรับ ASP .NET นี้การรีจิสเตอร์คอมโพเนนต์จะไม่มีอีกต่อไป และจะไม่มีปัญหาการล็อกไฟล์ DLL เกิดขึ้น ซึ่งในอดีตนั้น ถ้าเราจำเป็นต้องส่งมอบแอปพลิเคชันสำหรับเว็บไซต์ขนาดใหญ่ เราจำเป็นต้อง Shut down เซิร์ฟเวอร์ก่อนที่จะติดตั้งเวอร์ชันใหม่ลงไป ซึ่งสำหรับเว็บไซต์ขนาดใหญ่ นั้นเรื่องนี้จะมีความเป็นอยู่อย่างยาก แต่สำหรับ ASP .NET นั้นจะใช้งานไฟล์ XML เพื่อเก็บข้อมูลต่างๆ ซึ่งจะทำให้เราส่งมอบแอปพลิเคชันได้ง่ายขึ้น โดยถือปฏิบัติทั้งไดเรกทอรีเท่านั้น

นอกจากนี้ ASP .NET ยังสนับสนุนบราวเซอร์หลายๆ ตัวด้วยกัน ทำให้ปัญหาเรื่องเข้ากันได้ (Compatible) นั้นหมดลงไป และ ASP .NET ยังสนับสนุนการคอมไพล์โค้ดด้วย ทำให้เราไม่จำเป็นต้องห่วงในเรื่องของประสิทธิภาพการทำงานซึ่งเกิดขึ้นในเวอร์ชันก่อน ที่ใช้ภาษาสคริปต์วิ่ง จะต้องมีการแปลความตอนรันแอปพลิเคชัน นอกจากนี้เรายังสามารถคอมไพล์โค้ดคำสั่งได้ด้วย ซึ่งการคอมไพล์โค้ดจะช่วยซ่อนซอร์สโค้ดที่เราได้เขียนขึ้นมา ทำให้เมื่อเราต้องส่งมอบโปรแกรม เราก็ไม่ต้องกังวลในเรื่องของการเปิดเผยซอร์สโค้ดให้คนอื่นเห็นอีกต่อไป

สรุปตารางเปรียบเทียบระหว่าง ASP กับ ASP .NET

คุณลักษณะ	ASP	ASP .NET
นามสกุลของไฟล์	.asp	.aspx, .ascx, .asmx
การทำงานของโปรแกรม	คอมไพล์โค้ดด้วยตัว Interpreter	คอมไพล์โค้ดด้วยตัว JIT Compiler
การจัดการเรื่องสถานะของโปรแกรม	- มีการใช้คุกกี้ (Cookies) ที่ไคลเอนต์ - ไม่สามารถทำงานร่วมกันผ่านแพลตฟอร์มได้	- สามารถฝังลงใน URL ได้เลยโดยไม่ต้องใช้คุกกี้ (Cookies) - สามารถบันทึกข้อมูลที่ใช้ร่วมกันลงในฐานข้อมูล SQL ภายนอกได้
อัปเดตไฟล์ DLL Library	ต้องปิดเครื่องเว็บเซิร์ฟเวอร์ก่อน	สามารถคัดลอกไฟล์ DLL ลงในไดเรกทอรี bin ได้ทันที (ไม่ต้องมีการลงทะเบียน)

ตารางที่ 4 -1 เปรียบเทียบความแตกต่างระหว่าง ASP กับ ASP .NET

4.3 เซิร์ฟเวอร์คอนโทรล (Server Control)

การสร้างกราฟฟิกรินเทอร์เฟซ (Graphic Interface) ใน ASP เวอร์ชันก่อนๆ ต้องอาศัยแท็ก <input ...> ของ HTML เป็นหลักแต่เนื่องจากข้อจำกัดหลายๆ ประการของแท็กดังกล่าวทำให้เราไม่สามารถสร้างกราฟฟิกรินเทอร์เฟซในรูปแบบที่แปลกๆ ใหม่ๆ และขั้นตอนการเขียนโปรแกรมก็ค่อนข้างยุ่งยาก ดังนั้นในเวอร์ชัน .NET นี้ไมโครซอฟต์จึงได้นำเอาคอนโทรลที่เราคุ้นเคยกันดีในการสร้าง วินโดว์แอปพลิเคชัน เช่น TextBox, Label, Button เป็นต้น มาทำเป็นกราฟฟิกรินเทอร์เฟซบนเว็บฟอร์ม (แต่รูปแบบเดิมนั้นก็ยังสามารถใช้ได้อยู่) และสามารถเขียนโปรแกรมเพื่อจัดการบางอีเวนต์ที่สำคัญๆ ของคอนโทรลตัวนั้นๆ ได้ (แต่ทำได้เพียงบางอีเวนต์เท่านั้น ไม่ครบทุกอีเวนต์ที่มีในวินโดว์แอปพลิเคชัน)

ตัวอย่างโค้ดของคอนโทรลพื้นฐานที่ทำงานบนเซิร์ฟเวอร์

```
<asp : textbox id = text1 runat=server/>

text1.text = "Hello World"

<asp : checkbox id = check1 runat=server/>

check1.checked = True

<asp : button id = button1 runat=server/>

button1_Click()

<asp : DropDownList id = DropDownList1 runat=server/>

DropDownList1.SelectedItem.Text = "Hello"
```

4.4 วาริเดชั่นคอนโทรล (Validation Controls)

โดยทั่วไปการกรอกข้อมูลของผู้ใช้มักมีความผิดพลาดอยู่เสมอ ทั้งแบบที่ตั้งใจ และไม่ได้ตั้งใจ ในบางกรณีหากผู้ใช้กรอกข้อมูลที่ผิดพลาด อาจส่งผลให้โปรแกรมไม่สามารถทำงานต่อได้เลย วิธีการเดิมที่ใช้กัน คือ การนำข้อมูลที่ผู้ใช้กรอกเข้าไปตรวจสอบด้วยการเขียนโปรแกรมเองทั้งหมด แต่ใน ASP .NET นี้มีคอนโทรลที่มามีหน้าที่ในการตรวจสอบข้อมูลที่ผู้ใช้กรอกเข้ามาโดยเฉพาะ ว่าเป็นไปตามรูปแบบที่ต้องการหรือไม่ โดยที่เราไม่ต้องเขียนโปรแกรมให้ยุ่งยาก เพียงแต่กำหนดรูปแบบการตรวจสอบให้กับคอนโทรลเท่านั้น ก็สามารถใช้งานได้แล้ว

ตัวอย่างโค้ดของคอนโทรลวาริเดชั่น

```
<asp : RequiredFieldValidator ControlToValidate = "txtName"
ErrorMessage = "Please Enter Your Name" runat = "server"/>
```

4.5 ภาษาที่ใช้เขียน ASP .NET

ภาษาที่ใช้ในการพัฒนา ASP .NET นั้นนับได้ว่าเป็นจุดเปลี่ยนแปลงที่สำคัญที่สุดจุดหนึ่งของ ASP .NET จากเดิมที่เราเคยใช้กันแต่ VBScript มาคราวนี้ VBScript ได้ถูกแทนที่ด้วย VB .NET, C# และ Jscript ทั้งนี้เพื่อให้เราสามารถดึงความสามารถจากภาษาเหล่านี้มาใช้ได้อย่างเต็มที่

4.6 รู้จักกับ System.Web.UI Namespace

ในส่วนนี้จะอธิบายถึงคลาส ที่สำคัญที่อยู่ในเนมสเปซชื่อ System.Web.UI ในเฟรมเวิร์คของ ASP.NET เนมสเปซ System.Web.UI นั้นระบุคลาสและอินเทอร์เฟซที่ใช้ในการสร้างสิ่งต่างๆในคลาสเว็บฟอรม ที่สำคัญที่สุดในSystem.Web.UI คือคลาส Control ซึ่งบอกคุณสมบัติ เมธอด และ event ของแต่ละเซิร์ฟเวอร์คอนโทรลในเฟรมเวิร์คของเว็บฟอรม อีกคลาสที่สำคัญคือเพจ ซึ่งจะกล่าวถึงในลำดับต่อไป

4.6.1 คลาส Control

คลาส Control เป็นรากฐานของคอนโทรลทั้งหมด ตัวอย่างเช่น textbox หรือ button ก็คือคอนโทรล โดยทั่วไปคลาส Control จะมีฟังก์ชันและคุณสมบัติของส่วนติดต่อกับผู้ใช้ทุกตัว ทุกอย่างที่เราเห็นใน ASP.NET ก็เป็นคลาสที่สืบทอดมาจากแหล่งใดแหล่งหนึ่งทั้งสิ้น

4.6.1.1 คุณสมบัติของ Control

คลาส Control มีคุณสมบัติที่สำคัญดังต่อไปนี้ Control, ID, Parent, EnableViewState, Visible, Context และ ViewState คุณสมบัติของคอนโทรลเหล่านี้แทนลูกของ instance ของคอนโทรลคุณสมบัติของคลาสแม่จะบอกคลาส แม่ของคอนโทรล คุณสมบัติทั้งสองนี้ ทำให้มีลำดับชั้น(hierarchy)ของคอนโทรลในหน้าเว็บคุณสมบัติ ID ทำให้คอนโทรลถูกเข้าถึงจากการเขียนโปรแกรมได้โดยใช้ ID ตามด้วยจุดและชื่อคุณสมบัติหรือเมธอดอื่นๆที่เราต้องการจะเข้าถึงของคอนโทรลนั้น เช่น MyObjectId.propertyname และยังสามารถเขียนตัวจัดการกับเหตุการณ์ที่เกิดขึ้นกับคอนโทรลได้ด้วย

EnableViewState เป็น flag ที่บอกว่าคอนโทรลจะยังคงเก็บview ของสถานะมันไว้หรือไม่ เช่นเดียวกับ view สถานะทั้งหมดของคอนโทรลลูกของมัน คุณสมบัติของ Context ทำให้เราสามารถดึงข้อมูลของ HTTP request ได้

ViewState เป็นอินสแตนซ์ของคลาส StateBag ซึ่งใช้ในการเก็บชื่อและค่าของข้อมูลที่สามารถสร้างให้เข้าถึงได้โดยหลาย request สำหรับหน้าเพจเดียว ชื่อและค่าเหล่านี้คืออินสแตนซ์ของคลาส StateItem

4.6.1.2 เมธอดในคลาส Control

เมธอดของคลาส Control นั้นมีเป็นจำนวนมาก อย่างไรก็ตามเราจะกล่าวถึงเพียงเมธอดที่มีความสำคัญ

เมธอด DataBind

เป็นการโยกคอนโทรลเข้ากับแหล่งข้อมูล เมื่อเมธอด นี้ถูกเรียก แท็กของการผูกข้อมูล <%#%> จะถูกนำมาพิจารณาใหม่เพื่อให้ข้อมูลใหม่ถูกโยกเข้ากับตำแหน่งtagที่เหมาะสม

เมธอด CreateChildControls

จะถูกเรียกก่อนที่คอนโทรล compositional custom จะถูกจัดแสดงคอนโทรล compositional custom นั้นคล้ายกับคอนโทรล ActiveX คือมันจะจัดการสร้างคอนโทรลอื่นๆ ในการพัฒนาคอนโทรล custom เมธอดนี้สามารถถูก override เพื่อให้ผู้พัฒนาคอนโทรล custom สามารถสร้าง คอนโทรลลูก ก่อนทำการแสดงคอนโทรลไม่ว่าในการทำครั้งแรกหรือครั้งต่อไปก็ตาม

เมธอด Render

คล้ายกับ CreateChildControls เริ่มต้นใช้สำหรับพัฒนาคอนโทรล custom ผู้พัฒนาคอนโทรลจะ override เมธอดนี้ เพื่อแสดงตัวเนื้อหาในคอนโทรลผ่านทางพารามิเตอร์ HtmlTextWriter ที่เตรียมไว้

เมธอด SaveViewState และ LoadViewState

เก็บบันทึกและเรียกซ้ำสถานะของคอนโทรล คอนโทรลของเซิร์ฟเวอร์ คงสถานะของมันระหว่างการrequest ผ่านทางเมธอด นี้

4.6.2 คลาสเพจ

คลาสเพจนี้สืบทอดมาจากคลาส Control ซึ่งหมายความว่ามันสืบทอดมาทั้งคุณสมบัติเมธอด และ event ที่แสดงโดยคลาส Control นอกจากการสืบทอดแล้วคลาส เพจ ระบุคุณสมบัติเมธอด และ event ที่เจาะจงมากขึ้น สำหรับหน้าเว็บในเฟรมเวิร์คของ ASP.NET

ใน ASP.NET ทั้ง Application, Request , Respond , Server และ Session เป็นคุณสมบัติของคลาส เพจ นอกเหนือจากอ็อบเจกต์เหล่านี้คลาส เพจ ยังมีคุณสมบัติอื่นๆอีก เช่น Cache, ErrorPage, Postback, IsValid, Trace และ Validators

4.6.2.1 ตัวอย่างคุณสมบัติและเมธอดของคลาสเพจได้แก่

คุณสมบัติ Cache

ชี้ไปยังอ็อบเจกต์ cache ของขอบเขตของหน้าปัจจุบัน ในที่นี้ ทรัพยากรเช่นการติดต่อฐานข้อมูล (database connection)จะถูกเก็บเพื่อนำมาใช้ใหม่โดยไม่ต้องสร้างการติดต่อใหม่ หากว่า cache item นั้นยังไม่หมดอายุ

คุณสมบัติ ErrorPage

บอกหน้าที่จะถูกเรียกให้แสดงเมื่อมี error เกิดขึ้น อาจใช้ @Pageแทนได้

คุณสมบัติ IsPostBack

บอกว่าการเรียกหน้าเว็บเป็นการโหลดครั้งแรกหรือไม่ หาก IsPostBack มีค่าเป็น true แสดงว่าไม่ใช่การโหลดครั้งแรก จึงไม่ควรทำการ initialize หน้าเพจซ้ำ เพื่อทำให้ประสิทธิภาพดีขึ้น

คุณสมบัติ Validator

รวมกลุ่มของเซิร์ฟเวอร์คอนโทรลซึ่งสามารถกำหนดระยะเวลาการทำงานของตัวเองลงในคุณสมบัติ Validator ของหน้าเว็บ เมื่อหน้าเว็บใดต้องมีการกำหนดอายุของตัวเอง ก็จะกำหนดอายุของคอนโทรลทั้งหมดในหน้านั้นแล้วกำหนดค่าที่เหมาะสมให้แก่คุณสมบัติ IsValid

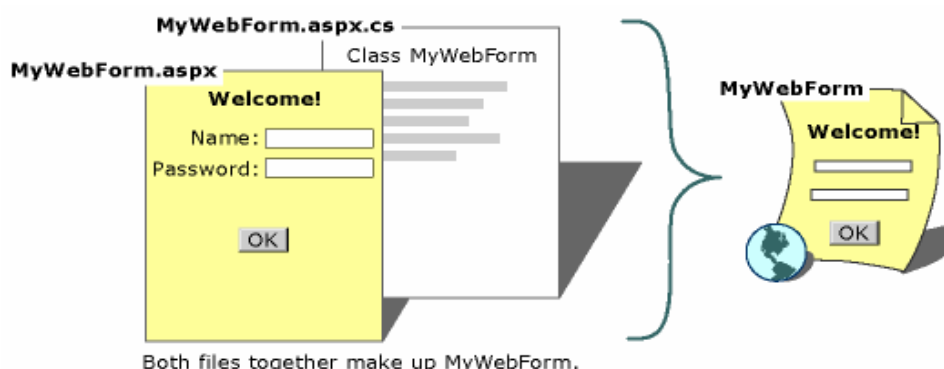
คุณสมบัติ Trace

อ้างอิงจากอ็อบเจกต์ TraceContext ซึ่งใช้บอกค่าเตือนหรือข้อความบอกความผิดพลาดที่เกิดขึ้น สามารถกำหนดให้ทำการตรวจสอบหรือไม่ทำก็ได้โดยกำหนดใน web.config ซึ่งเป็นเท็กซ์ไฟล์ที่เก็บค่าขณะรันไทม์ของแอปพลิเคชันของASP.NET

4.7 การพัฒนาเว็บแอปพลิเคชันด้วย ASP.NET

4.7.1 ส่วนประกอบคอมโพเนนต์ของ Web Form

ใน Web Forms pages นั้น การเขียน program ของ user interface แบ่งเป็นสองส่วนคือ visual Component และ logic ซึ่งก็คล้ายกับการที่ Visual Basic มีการแบ่งระหว่างส่วนของ form และส่วน code เบื้องหลังที่ form นั้นทำงานด้วยนั่นเอง ส่วนแสดงผลที่มองเห็นได้นั้นก็คือ Web Form page ซึ่งประกอบด้วยไฟล์ ที่มี HTML หรือ ASP.NET server control. Web Form page ทำงานเหมือนเป็น container สำหรับ static text และส่วนควบคุมที่เราต้องการแสดงผล ในส่วน logic ของ Web Form page ประกอบด้วย โค้ด ที่เรา สร้างขึ้นเพื่อสื่อสารกับ form logic การเขียนโปรแกรม จะอยู่แยกกับส่วน file ของ user interface ซึ่งไฟล์นี้จะเปรียบเสมือน โค้ดที่อยู่เบื้องหลังการทำงาน สามารถเขียนด้วยภาษา Visual Basic หรือ Visual C#



รูปที่ 4-1 ส่วนประกอบของเว็บฟอร์ม

ไฟล์ที่ประกอบด้วยคลาสต่างๆและ เบื้องหลังการทำงาน สำหรับ Web Form ทั้งหมดจะถูก คอมไพล์ เป็น ไฟล์ dynamic-link library (.dll) page ที่ เป็น .aspx ก็จะถูก compile เช่นกัน แต่มีกระบวนการที่ต่างกัน ในครั้งแรกที่ ผู้ใช้ เรียก file หน้า .aspx ASP.NET จะสร้างไฟล์ .NET class มาเป็นตัวแทนของ page โดยอัตโนมัติ แล้ว คอมไพล์ เป็น ไฟล์ .dll ตัวที่สอง class ที่ถูกสร้างสำหรับ .aspx page สืบทอดคุณสมบัติมาจาก class ที่เป็น โค้ดการทำงานเบื้องหลังที่ถูก compile ไปแล้วก่อนหน้านี้ เมื่อผู้ใช้ร้องขอ URL ของ web page ไฟล์ .dll จะ run บน server และสร้างผลเป็น HTMLแบบไดนามิกสำหรับ page ที่เรียก

4.7.2 วงจรชีวิตของ Web Form

ใน ASP เว็บเพจจะเริ่มต้นวงจรชีวิตเมื่อไคลเอนต์เริ่มทำการร้องขอเพจที่เจาะจงมา IIS ทำการกระจายคำและทำการใช้งานสคริปต์บน ASP เพจ เพื่อให้เป็นเนื้อความ HTML ในที่สุดการทำการแปลงก็จะสิ้นสุดอายุของเพจนั่นก็จะสิ้นสุด ถ้าคุณมีฟอร์มที่ส่งข้อมูลกลับไป ASP เพจก็จะเริ่มกระบวนการใหม่เหมือนกับ ASP กระทำคำร้องใหม่ โดยไม่ต้องรู้เกี่ยวกับขั้นตอนก่อนหน้าเลย การส่งผ่านข้อมูลกลับไปที่เพจเริ่มต้นสำหรับการประมวลผลก็เป็นการถูกอ้างแบบโพสต์แบ็ค (postback) ด้วย

ใน ASP.NET มีหลายสิ่งที่แตกต่างกันเล็กน้อย เพจยังคงเริ่มที่การร้องขอของไคลเอนต์อย่างไรก็ตาม มันจะยังคงอยู่เรื่อยๆเท่าที่ไคลเอนต์ยังคงติดต่อกับเพจอยู่ เพื่อให้ง่ายขึ้นอาจจะกล่าวได้ว่าเพจยังคงอยู่ไปเรื่อยแต่ในความเป็นจริงมีเพียงขั้นตอนการดู(view)ของเพจระหว่างการร้องขอเท่านั้น ขั้นตอนการดูเท่านั้นที่อนุญาตให้ควบคุมบนเซิร์ฟเวอร์ที่จะปรากฏว่า ถ้ามันยังคงครองอีเวนต์ของเซิร์ฟเวอร์ เราสามารถค้นหาขั้นตอนการโพสต์แบ็คของเพจผ่านคุณสมบัติ IsPostBack ของเพจอ็อบเจกต์และการกำหนดค่าเริ่มต้นใหม่ การครองอีเวนต์ระหว่างการโพสต์แบ็คมันก็คือการทำ ASP.NET แตกต่างกว่าการพัฒนา ASP ตามแบบธรรมชาติอย่างมาก

จากตัวอย่างเราทำการขยายส่วนต้น เช่นการครองโพสต์แบ็ค เมื่อทำการโหลดอีเวนต์ที่ถูกถือครองครั้งแรก เราจะทำการย้ายข้อมูลลงใน drop-down list box หลังจากนั้นเราจะระบุเวลาของอีเวนต์ที่เพิ่มขึ้นโดยไม่มีกรโหลดข้อมูลเข้ามาใหม่เท่านั้น ตัวอย่างนี้แสดงให้เห็นว่าเซิร์ฟเวอร์อีเวนต์แฮนด์เลอร์(handler)ที่ชื่อ handlerButtonClick เป็นขอบเขตของอีเวนต์ที่ชื่อ ServerClick ของปุ่ม

```
<html>

<head><title>Testing Page Event</title></head>

<body>

<script language="C#" runat=server>

    void Page_Init(Object oSender,EventArgs oEvent){

        labelInit.Text = DateTime.Now.ToString( );
```



```

    }

    void Page_Load (Object oSender, EventArgs oEvent) {
        labelInit.Text = DateTime.Now.ToString ();
        if (!IsPostBack){
            selectCtrl.Items.Add("Acrua");
            selectCtrl.Items.Add("BMW");
            selectCtrl.Items.Add("Cadillac");
            selectCtrl.Items.Add("Mercedes");
            selectCtrl.Items.Add("Porsche");
        }else{
            labelLoad.text += "(Postback)";
        }
    }

    void handleButtonClick(Object oSender, EventArgs oEvent){
        labelOutput.Text = "You've selected: "+ selectCtrl.Value;
        labelEvent.Text = DateTime.Now.ToString ();
    }
</script>
<form runat = server>
    Init Time: <asp:Label id=labelInit runat=server/><br/>
    Load Time: <asp:Label id=labelLoad runat=server/><br/>
    Event Time: <asp:Label id=labelEvent runat=server/><br/>
    Choice: <select id=selectCtrl runat=server/><br/>
    <asp:Label id=labelOutput runat=server/></select><br/>
    <input type=button value=update
        onServerClick="handleButtonClick" runat=server />
</form>
</body>
</html>

```

วงจรชีวิตของเว็บฟอร์ม ประกอบด้วยสามขั้นตอนหลัก การกำหนดรูปแบบ การถือครองอีเวนต์ และการสิ้นสุด โดยขั้นตอนเหล่านี้จะข้ามผ่านคำร้องขอที่เหมือนกันซึ่งแข่งกับนโยบาย serving-one-page-at-a-time ที่พบใน ASP

4.7.2.1 การกำหนดรูปแบบ

ขั้นตอนการกำหนดรูปแบบนี้ โหลดอีเวนต์ของเพจจะถูกยกขึ้นมันเป็นงานของคุณในการถือครองอีเวนต์นี้เพื่อติดตั้งเพจคุณ เพราะว่าโหลดอีเวนต์จะถูกยกขึ้นเมื่อการควบคุมพร้อมแล้วงานคุณตอนนี้คือการอ่านและทำการอัปเดตคุณสมบัติการควบคุมเหมือนเป็นส่วนของการติดตั้งเพจ ในตัวอย่างโค้ดข้างต้น เราจะถือครอง โหลดอีเวนต์เพื่อย้ายข้อมูลบางอย่างลง drop-down list เราอัปเดตข้อความของการควบคุม labelLoad เพื่อแสดง เวลาโหลดอีเวนต์ที่เกิดขึ้น ในแอปพลิเคชันของคุณคุณจะต้องข้อมูลที่เป็นไปได้จาก ฐานข้อมูลและทำการหาค่าเริ่มต้นฟอร์มฟิลด์ด้วยค่าฟิลด์

คุณสมบัติ IsPostBack ของเพจแสดงว่าเพจแรกได้ถูกโหลดหรือมันเป็นการโพสต์แบคเช่นถ้าคุณควบคุมเนื้อหารายการของข้อมูลคุณจะต้องโหลดการควบคุมเพจแรกที่ถูกโหลดโดยการตรวจสอบคุณสมบัติ IsPostBackของเพจนี้นั้นเมื่อ IsPostBack มีค่าเป็นจริงคุณจะได้รู้ว่ารายการควบคุมอ็อบเจกต์ได้ถูกโหลดด้วยข้อมูลเรียบร้อยแล้ว ไม่จำเป็นที่ต้องย้ายข้อมูลในรายการใหม่ตัวอย่างโค้ดข้างต้นเราจะข้ามการย้ายข้อมูลของ drop-down และแสดงเพียง สตริงว่า “(postback)”

คุณอาจจะต้องการทำการผูก (bind) ข้อมูลและกำหนดค่าใหม่ data-binding expressions ในตอนแรก และลำดับต่อมาของเพจนี้นี้

4.7.2.2 การถือครองอีเวนต์

ในขั้นตอนกลางนี้ ฟังก์ชัน server event-handlingของเพจจะถูกเรียกเป็นผลของบางอีเวนต์เป็นการถูกเรียกให้ทำงานจากฝั่งไคลเอนต์ อีเวนต์เหล่านี้มาจากการควบคุมที่คุณแทนที่บนเว็บฟอร์ม

4.7.2.3 การจบการทำงาน

ที่ขั้นตอนนี้เพจจะสิ้นสุดการแปลงและถูกกำจัดทิ้งแล้วสิ่งที่คุณต้องรับผิดชอบคือการทำความสะอาดไฟล์แฮนด์เลอร์ ยกเลิกการติดต่อกับฐานข้อมูลและปล่อยอ็อบเจกต์ถึงแม้ว่าคุณจะเชื่อมั่นการทำงานของตัวกำจัดขยะ(garbage collection)ของ CLR ก็ตามแต่เราขอยืนยันอย่างชัดแจ้งว่าควรกำจัดความสะอาดด้วยตัวเองเพราะตัวกำจัดขยะทำงานเป็นระยะๆเท่านั้น บนการทำงานหนักๆถ้าตัวกำจัดขยะไม่เหมาะสมจะมีทรัพยากรที่ไม่ได้คืนให้ระบบทำให้หน่วยความจำหมดไปได้และทำให้ระบบคุณค้างไป

เราสามารถทำความสะอาดตัวอย่างข้างต้นได้ด้วยการอันโหลดอีเวนต์แฮนด์เลอร์ดังที่จะแสดงต่อไปนี้เป็นเพราะว่า ไม่มีแสดงในตัวอย่างเราจะแสดงเฉพาะฟังก์ชันเป็น เทมเพลต

```
Void Page_Unload(Object oSender,EventArgs oEvent){  
    //cleaning up code here  
}
```